

Research Project in Computer Systems Engineering

Final Report

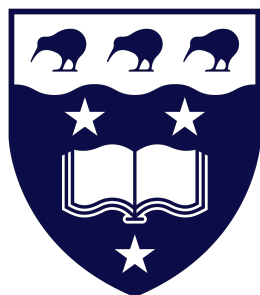
Reconfigurable Neural Processing Unit (NPU) for Energy-Efficient AI at the Edge

Pratham Chhabra

Summer Research Assistant

Supervisor: Dr Morteza Biglari-Abhari

April 7, 2026



Waipapa
Taumata Rau
**University
of Auckland**

RECONFIGURABLE NEURAL PROCESSING UNIT (NPU) FOR ENERGY-EFFICIENT AI AT THE EDGE

Pratham Chhabra

ABSTRACT

Presented in this Research Project Report, commissioned as part of Electrical, Computer, and Software Engineering at the University of Auckland, is the design and implementation of a Reconfigurable Dual-Core Neural Processing Unit (NPU) for Energy-Efficient Artificial Intelligence at the edge. Building upon a prior simulation-based implementation of a coordinated row-and-column sparsity-stripping algorithm and an 8x8 output-stationary systolic array on a Cyclone V FPGA, this work extends the design to a hardware-verified, multi-model inference system deployed on an Intel Arria 10 SoC FPGA. The proposed architecture addresses the limitations of static hardware accelerators by instantiating two physically distinct compute cores within a unified NPU System Wrapper: an INT16 32x32 DSP-based core for high-precision dense tiles, and an INT8 16x16 LUT-based core for sparse, dimensionally reduced tiles. A Python-based software stack using the OpenVINO Intermediate Representation extracts per-layer activation and weight tensors from four CNN architectures: VGG16, ResNet50, MobileNetV2, and AlexNet. It applies quantisation and the im2col transformation, and executes the sparsity-stripping algorithm to produce compacted tile dimensions M, N, and K. These dimensions drive a runtime routing decision that selects the optimal core on a per-tile basis without hardware modification. A TCL automation layer manages JTAG-to-Avalon tile dispatch, enabling unsupervised multi-layer inference across all evaluated models. Hardware synthesis on the Arria 10 confirmed timing closure at 70 MHz with a setup slack of 5.723 ns, using 69,367 ALMs and 1,287 DSP blocks. Across all four models, the dual-core system achieves an average cycle reduction of 30-55% per layer compared to a dense single-core baseline, with Config 1 demonstrating normalised area efficiency scores up to 28x higher than Config 0 for sparse workloads. The results confirm that the co-design approach generalises across diverse CNN architectures, reveals model-specific computational signatures, and validates the reconfigurable dual-core strategy as a practical solution for energy-efficient edge inference on FPGAs.

Table of Contents

Acknowledgements	iv
Glossary of Terms	v
Abbreviations	v
1 Introduction	1
2 Prior Work	1
3 Literature Review Relevant to Current Design	3
3.1 Multi-Precision and Reconfigurable Architectures for CNN Inference	3
3.2 Host-to-FPGA Communication Interfaces for Inference Pipelines	4
3.3 Model-Agnostic Inference Deployment via OpenVINO	5
4 Methodology	5
4.1 Architectural Design and Pipeline	5
4.1.1 System Dataflow	6
4.1.2 Host-to-FPGA Pipeline	6
4.1.3 Hardware Architecture	6
4.2 OpenVINO Software Stack on Host Computer	7
4.3 TCL Automation	7
4.4 Reconfigurable Dual Core NPU Architecture	9
4.4.1 Dual-Core Precision Orchestration	9
4.4.2 Finite State Machine	11
4.5 Verification and Validation Strategies	13
4.5.1 Pre-Silicone Performance Profiling and Software Validation	13
5 Results	14
5.1 NPU Performance Table	14
5.2 Results Collected from different CNN models	15
6 Discussion	20
7 Future Work	23
7.1 HPS-Based Deployment with On-Device OpenVINO	23
7.2 PCIe-Based High-Bandwidth Host Interface	23
8 Conclusion	24
References	25

Acknowledgements

I would like to express my appreciation towards Dr Morteza Biglari-Abhari and for his unmatched support for this project throughout the year 2025 and during the summer for this the summer research period.

I would also like to express my appreciation to my part IV project partner, Kristelle Sampang and supervisor, Dr Maryam Hemmati, for their support that helped build the foundations of this project during my final year of studies.

Glossary of Terms

Term	Definition
Activation Sparsity	A form of sparsity in neural networks caused by activation functions (like ReLU) setting negative outputs to zero. This is dynamic and changes with each input image.
Control Unit	The hardware component responsible for orchestrating data flow, generating timing signals (staggering), and managing the operation of the systolic array.
Edge (AI at the Edge)	The practice of running AI inference directly on local, resource-constrained devices rather than in the cloud.
Fine-Grained Sparsity	See Unstructured Sparsity.
Latency	The time, measured in clock cycles, required to complete a computational task.
Processing Element (PE)	The fundamental building block of a systolic array, typically containing a Multiply-Accumulate (MAC) unit and registers.
Pruning	An optimisation technique that removes redundant or insignificant weights from a neural network to reduce model size and computational complexity.
Quantisation	An optimisation technique that reduces the numerical precision of a model's weights and activations (e.g., from 32-bit float to 8-bit integer) to save memory and simplify hardware.
Sparsity	The property of a matrix containing a large proportion of zero-valued elements.
Staggering	The process of delaying the input of data and weight elements into the systolic array boundaries to ensure they meet at the correct PE at the correct time.
Structured Sparsity	Sparsity that occurs in a regular pattern, such as entire rows or columns of a matrix being zero. This is the type of sparsity targeted by this project's algorithm.
Systolic Array	A parallel computing architecture consisting of a grid of Processing Elements (PEs) designed for high-throughput, pipelined operations like matrix multiplication.
Throughput	A measure of computational performance, indicating the number of operations that can be performed per unit of time (e.g., GOPs/sec).
Tiling	The method of breaking down large matrices into smaller, fixed-size blocks (tiles) to be processed by a hardware accelerator of a fixed size (e.g., 8x8).
Unstructured Sparsity	Sparsity where zero-valued elements are randomly and irregularly distributed throughout a matrix.
Weight Sparsity	A form of sparsity in neural networks that is typically static, resulting from model optimisation techniques like pruning.

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence

Abbreviation	Full Form
ALM	Adaptive Logic Module
CNN	Convolutional Neural Network
DPR	Dynamic Partial Reconfiguration
DSP	Digital Signal Processor
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GEMM	General Matrix-Matrix Multiplication
GOPs	Giga Operations Per Second
HPS	Hard Processor System
IP	Intellectual Property
IR	Intermediate Representation
JTAG	Joint Test Action Group
LUT	Lookup Table
MAC	Multiply-Accumulate
NPU	Neural Processing Unit
PCIe	Peripheral Component Interconnect Express
PE	Processing Element
ReLU	Rectified Linear Unit
SoC	System on Chip
TCL	Tool Command Language
TPU	Tensor Processing Unit

List of Figures

Figure 1	Complete System Diagram	6
Figure 2	High-Level Diagram of the NPU System Wrapper With Encapsulated States	9
Figure 3	Algorithmic State Machine Diagram Of The FSM	11
Figure 4	Full Execution Timing Diagram of the NPU	12
Figure 5	Timing Diagram of the Result Fetching	13
Figure 6	Theoretical Maximum Efficiency Achieved for each Tile and precision flexibility	14
Figure 7	Theoretical Average efficiency achieved for each Quantisation precision	14
Figure 8	Average Latency per Core Configuration: Config 0 (32x32 DSP) vs. Config 1 (16x16 LUT).	15
Figure 9	NPU vs. CPU Cycles per Layer comparison.	15
Figure 10	Latency Saved between Baseline and 2-Core NPU.	16
Figure 11	Impact of Sparsity on Computational Latency (Sparsity vs. Cycles). . . .	16
Figure 12	Dense-Equivalent Throughput (GOPs) vs. Sparsity Level.	17
Figure 13	Normalized Area Efficiency (GOPs/Area) per Core Configuration. . . .	17
Figure 14	Power to Silicon Area Efficiency Score for Each Layer.	18
Figure 15	Total Energy Consumption (mJ) per Layer.	18

1. Introduction

The deployment of Convolutional Neural Networks on resource-constrained edge devices presents a fundamental tension between computational demand and hardware efficiency. Fixed-architecture accelerators, while effective for dense workloads, fail to exploit the structured sparsity that arises naturally in optimised neural networks, resulting in wasted computational cycles and energy on operations whose outcomes are known in advance to be zero.

This report is a direct continuation of prior work, which established the theoretical basis for a hardware-software co-design approach to this problem. That work designed and simulated a coordinated row-and-column removal algorithm paired with a reconfigurable 8x8 output-stationary systolic array on a Cyclone V FPGA, demonstrating that structured sparsity could be translated into measurable latency reduction through software pre-processing alone. However, that implementation remained simulation-based, evaluated only a single model on a single layer, and lacked a runtime data transfer pipeline or any mechanism for precision adaptation.

The current work directly addresses these limitations. It extends the design to a dual-core reconfigurable NPU deployed on an Intel Arria 10 SoC FPGA, introduces a heterogeneous INT16/INT8 compute architecture with per-tile runtime routing, integrates a full OpenVINO-based inference pipeline supporting four CNN architectures, and establishes hardware-verified operation through a TCL-automated JTAG dispatch system. The prior work, literature context, and theoretical foundations are established in Section 2 and 3, respectively.

2. Prior Work

This section provides a review of prior work relevant to the current implementation. It establishes the theoretical basis for the hardware-software co-design approach and contextualises the contributions of this work relative to the simulation-based implementation described in [1]

CNNs are mainly used for vision-based inference applications. Their computational cost is concentrated in the convolutional layers, which can account for up to 90% of total inference time [2], [3]. The convolution operation is mathematically equivalent to a General Matrix-Matrix Multiplication (GEMM) problem, achieved through the im2col transformation, which reshapes the input feature map into a 2D activation matrix and stacks the convolutional filters into a weight matrix [4]. This reformulation allows the entire convolutional layer to be expressed as a single matrix product $C[M \times N] = A[M \times K] \times B[K \times N]$, where M and N represent the spatial and filter output dimensions, respectively, and K is the shared inner dimension capturing the computational depth of the operation [5], [6]. This mapping serves as the foundation for all hardware acceleration in this work.

The rectified Linear Unit (ReLU) activation function, applied after each convolutional layer, sets all negative values to zero and is the primary source of activation sparsity in CNN inference [7] [8]. Because ReLU is applied after every layer, the activation matrices fed into subsequent layers can carry large proportions of zero-valued elements, particularly in deeper layers when feature maps become increasingly selective. This sparsity is dynamic, so its distribution changes with each input image, but the aggregate density can be predicted to be high for natural image inputs.

Systolic arrays provide an exceptionally well-suited architecture for executing GEMM operations in hardware, introduced by Kung and Leiserson [9], [10]. A systolic array consists of a grid of Processing Elements (PEs), each containing a Multiply-Accumulate (MAC) unit with internal

registers. Data and Weights are streamed in a pipelined, rhythmic fashion through the PE grid, where data propagates horizontally, and weights propagate vertically, so operands meet at the correct PE and clock cycle. This lockstep data movement eliminates the need for repeated memory accesses, reducing memory bandwidth by a factor proportional to the array width and enabling high sustained throughput.

The output-stationary dataflow variant, adopted in this work, keeps partial sums stationary within each PE’s accumulator register throughout the computation. This simplifies the PE design by eliminating the need for inter-PE partial-sum routing and allows each PE to independently accumulate its contribution to the output matrix. The effectiveness of the systolic array for CNN inference is well established in commercial hardware; Google’s Tensor Processing Unit (TPU) demonstrated 83x and 29x improvements in performance-per-watt over CPU and GPU baselines, respectively [11]. The prior work implemented an 8x8 output-stationary systolic array in VHDL, verified using ModelSim, and demonstrated that the architecture correctly executes matrix multiplication with a latency of $M + N + K - 2$ clock cycles [1].

While systolic arrays provide high throughput for dense workloads, their lockstep dataflow becomes inefficient when sparsity is present. When activation or weight values are zero, the corresponding PE still performs a MAC operation, consuming cycles and energy on a computation whose result is known in advance to be zero. Studies report that unstructured sparsity can reach 90% in CNNs following ReLU activation [12], making this an acute inefficiency for practical inference workloads.

Existing approaches to this problem divide broadly into hardware-centric and co-design methodologies. The Sparse-TPU pre-packs sparse matrices before dispatch to the systolic array, achieving up to 16x performance improvement and 4.39x lower energy consumption for 8-bit integer workloads [13]. The SCNN accelerator restructures the dataflow entirely, operating on compressed sparse formats and computing outer products of non-zero operands [14]. While SCNN achieves a 2.7x performance increase over a dense baseline, its irregular data access patterns introduce substantial hardware complexity that is difficult to reconcile with the simplicity goals of edge deployment. The sense accelerator takes a structured sparsity approach, using load balancing across weight-sparse matrices to achieve up to 2.5x speedup [12]. A co-design approach pairing structured pruning with matched systolic array dimensions achieves up to 44% speedup with reduced hardware overhead [15].

Simpler hardware-level approaches, such as fine-grained zero-skipping, disable the MAC operation when an operand is zero, achieving 83.8% MAC utilisation and 13.6 TOPs/W energy efficiency [16]. However, zero-skipping alone cannot reduce latency, since zero-valued data still occupies time slots as it traverses the array. An FPGA prototype implementing structured zero-skipping at the tile level achieved 51.96 GOPS/W but similarly could not reduce per-tile latency [17].

The prior work [1] identified this gap and proposed a coordinated row- and column-removal algorithm operating at the software pre-processing stage. By identifying rows of the activation matrix and columns of the weight matrix that are entirely zero-valued, and enforcing the corresponding removal in both matrices to preserve the mathematical validity of the GEMM operation, the algorithm produces compacted dense matrices of reduced dimensions M , N , K . These reduced dimensions are passed to the hardware control unit, which reconfigures the systolic array to execute only the necessary computation. Simulation results demonstrated a 60.87% average reduction in computational latency per tile across a 156-image dataset, with a theoretical peak throughput of 1.478 GOPs/sec at the reported synthesis frequency of 289.27 MHz on a Cyclone V FPGA.

Despite the promising simulation results, the prior implementation had several fundamental limitations that prevented it from serving as a hardware-verified inference system. First, all performance metrics were derived entirely from ModelSim simulations and Python estimation functions, rather than from physical hardware measurements. The synthesis frequency of 289.27MHz was reported directly from the Quartus timing analyser without the design ever being clocked at that rate in practice. Second, the hardware interface was minimal: input matrices were preloaded via Memory Initialisation Files (MIFs) at synthesis time, and computation was initiated via a push button. There was no runtime data transfer pipeline capable of processing sequential tiles from an inference workload. Third, evaluation was limited to a single configuration: an 8x8 INT8 systolic array that processed only the first convolutional layer of AlexNet from a single image. No multi-layer or multi-model inference was demonstrated. Additionally, the systolic array operated at a single fixed precision, without any mechanism for selecting a precision model based on the characteristics of the tile being processed.

3. Literature Review Relevant to Current Design

3.1 Multi-Precision and Reconfigurable Architectures for CNN Inference

A fundamental challenge in deploying CNN inference accelerators on FPGAs is the computational demand across network layers. Convolutional layers at different depths exhibit different arithmetic intensity, sparsity distributions, and precision requirements, meaning a single static hardware configuration may or may not simultaneously optimise all three parameters across a full inference pass. The comprehensive survey by Fata et al. identifies this as an unresolved problem in low-cost FPGA accelerator design, noting that none of the systems surveyed achieved a balanced score across throughput, accuracy, power consumption, latency, and energy efficiency simultaneously [18]. The survey categorises reconfigurability approaches into Dynamic Partial Reconfiguration (DPR), mode switching, and elastic on-chip buffering, and concludes that while DPR offers genuine runtime adaptability, its overhead in routing complexity, bitstream management, and control logic consistently suppresses throughput relative to static designs. Irmak et al., cited within the survey, demonstrate that a DPR-based accelerator achieves a $7\times$ improvement in frame rate over non-reconfigurable baselines, yet reports only 40.71 GOPS due to the reconfiguration latency penalty [18]. This throughput gap between what DPR promises in flexibility and what it delivers in sustained computation motivates investigation into alternative reconfigurability strategies that avoid physical fabric swapping. The resource efficiency trade-off between the two principal FPGA implementation media is well-characterised in this context: Bharathi and Shirur demonstrate that at 8-bit precision, LUT-based MAC structures consume significantly less area and logic power than DSP-based equivalents, while this advantage diminishes at higher bit-widths, where DSP blocks become the more efficient choice [19].

One alternative to DPR is multi-precision quantisation, where the accelerator hardware supports multiple fixed-point precisions simultaneously and selects between them based on workload characteristics. Cao et al. propose HMSA, a heterogeneous mixed-precision systolic array accelerator on FPGA that combines DSP-based and LUT-based MAC units, with hardware data paths configured prior to deployment through a resource-performance cost model [20]. Their analysis identifies that existing mixed-precision accelerators fail to fully exploit the bit-width headroom available in DSP blocks, and that residual LUT logic remains idle when only DSP-based PEs are deployed, causing practical inference throughput to fall well below theoretical peak performance. Nevertheless, while the hardware can be reconfigured between deployments, precision selection is not driven by runtime workload characteristics during inference. Shao et al. similarly propose a configurable CNN accelerator supporting both INT16 and INT8

quantisation on the same systolic array, where switching between precisions is governed by a pregenerated microinstruction sequence compiled ahead of inference [21]. This confirms that per-layer precision flexibility is achievable without full recompilation, but the precision assigned to each layer is determined statically rather than by measured runtime data characteristics such as sparsity. Neither work simultaneously deploys two physically distinct compute cores, nor uses tile-level sparsity as a live signal to route individual tiles to an appropriate precision path at runtime, leaving this combination as an open area of investigation.

3.2 Host-to-FPGA Communication Interfaces for Inference Pipelines

The interface between a host processing environment and an FPGA accelerator determines the practical ceiling on end-to-end inference throughput, independently of how efficient the compute fabric itself is. This has been identified as a persistent bottleneck in hardware-software co-design literature: Parameshwara and Mokashi report that even in a system using high-bandwidth AXI interconnects operating at 850 MB/s, DMA transfer overhead accounts for 15% of total accelerated latency and memory bandwidth stalls for a further 12%, meaning that over a quarter of total system latency originates at the communication boundary rather than the compute core [22]. This finding underscores that the interface selection is not a secondary concern but a primary architectural decision that must be explicitly addressed in any co-design system.

At the high-bandwidth end of the available interface spectrum, PCIe streaming represents the most capable host-to-FPGA data pathway currently available on discrete FPGA platforms. Manavar et al. evaluate PCIe streaming using the XDMA IP on a Xilinx Alveo U280, demonstrating that configuring the interface in streaming mode eliminates memory copy overhead entirely by passing data directly to the FPGA fabric at a rate bounded only by the synthesis frequency [23]. Their implementation achieves 65 million inferences per second at batch sizes two orders of magnitude smaller than a GPU, achieving equivalent peak throughput, while consuming $25\times$ less energy than a CPU baseline. The authors characterise that this performance level requires the hardware design to maintain an initiation interval of one, and that where this condition is not met, the PCIe bandwidth advantage over memory-mapped transfer diminishes substantially. This establishes a clear roofline constraint: the communication interface only ceases to be the bottleneck when the compute fabric can match its throughput cycle-for-cycle.

For SoC-based FPGAs such as the Arria 10, an alternative high-bandwidth pathway is available through the Hard Processor System. The Arria 10 integrates a dual-core ARM Cortex-A9 HPS connected to the FPGA fabric via AMBA AXI-3 compliant bridges that support simultaneous read and write transactions at data widths of 32, 64, and 128 bits, with direct access to the DDR4 memory subsystem [24]. This architecture means that a Linux-capable inference stack running on the HPS cores can dispatch tile data to the NPU accelerator via the H2F high-speed bridge without any external cable latency, serial protocol overhead, or driver-level bottlenecks. Guerrouj et al. identify HPS-based deployment as the natural production pathway for OpenVINO-based inference on the Arria 10 platform, noting that the compute-intensive sub-graphs of a CNN can be dispatched to the FPGA fabric while the HPS handles unsupported layer types as a fallback CPU, with the high-speed bridge providing the interconnect between the two [25].

Despite the availability of these high-bandwidth pathways, the existing literature does not address the intermediate problem of how a hardware-verified inference system should be architected when neither PCIe nor a functional HPS Linux stack is immediately available during the hardware bring-up phase. Existing works either assume production-grade interfaces from the outset and report throughput results accordingly, or rely entirely on simulation without deploying to

physical hardware. The gap is the absence of a systematic treatment of hardware verification as a distinct pipeline stage with its own interface requirements, where bit-accurate cross-validation of quantised, sparsity-stripped tile outputs against a software reference is the primary objective rather than throughput maximisation, and where the communication bottleneck is explicitly measured and reported as a known constraint with a defined upgrade path.

3.3 Model-Agnostic Inference Deployment via OpenVINO

The selection of a model deployment framework has significant implications for the generalisability of an inference pipeline, particularly when the objective is to evaluate accelerator performance across multiple CNN architectures without re-engineering the data extraction and quantisation logic for each model. Framework-specific approaches, such as direct PyTorch hook-based activation capture, are tightly coupled to a particular model’s internal graph structure and cannot be readily extended to models from different families without substantial code modification.

Guerrouj et al. evaluate the inference performance of ten classification models and nine object detection models on a heterogeneous Arria 10 FPGA/CPU architecture using the OpenVINO toolkit, reporting an average $3.6\times$ speedup for classification models on the FPGA/CPU heterogeneous configuration relative to a CPU-only baseline [25]. Their analysis describes the two-component OpenVINO pipeline: the Model Optimiser, which converts pre-trained models from frameworks including Caffe and TensorFlow into an Intermediate Representation (IR) consisting of an .xml topology file and a .bin weight file; and the Inference Engine, which reads the IR and dispatches compute-intensive sub-graphs to the FPGA fabric via the Deep Learning Accelerator plugin while falling back to the CPU for unsupported layer types. The authors observe that models containing activation functions or layer types not supported by the FPGA bitstream execute on the fallback CPU, and that this heterogeneous execution can introduce data transmission overhead between the two devices that partially offsets the FPGA speedup.

The broader capability of the OpenVINO IR representation to provide programmatic access to per-layer metadata, including kernel dimensions, stride, padding, and weight tensors via direct graph node inspection, makes it suitable as a model-agnostic extraction layer for accelerator research. However, the existing literature on OpenVINO-based FPGA deployment, including Guerrouj et al., evaluates throughput and latency at the full-model level and does not exploit the IR graph structure for per-layer, per-tile sparsity analysis or for dynamic routing decisions based on tile-level data characteristics. This represents a gap between OpenVINO’s documented capability as a graph inspection tool and its use in the literature as a black-box inference engine.

4. Methodology

4.1 Architectural Design and Pipeline

The NPU system architecture is designed as a hardware-software co-design pipeline that standardises the flow from a high-level deep learning framework to cycle-accurate FPGA execution. The pipeline is divided into three primary stages: Software Characterisation, Automated Orchestration, and Hardware Acceleration.

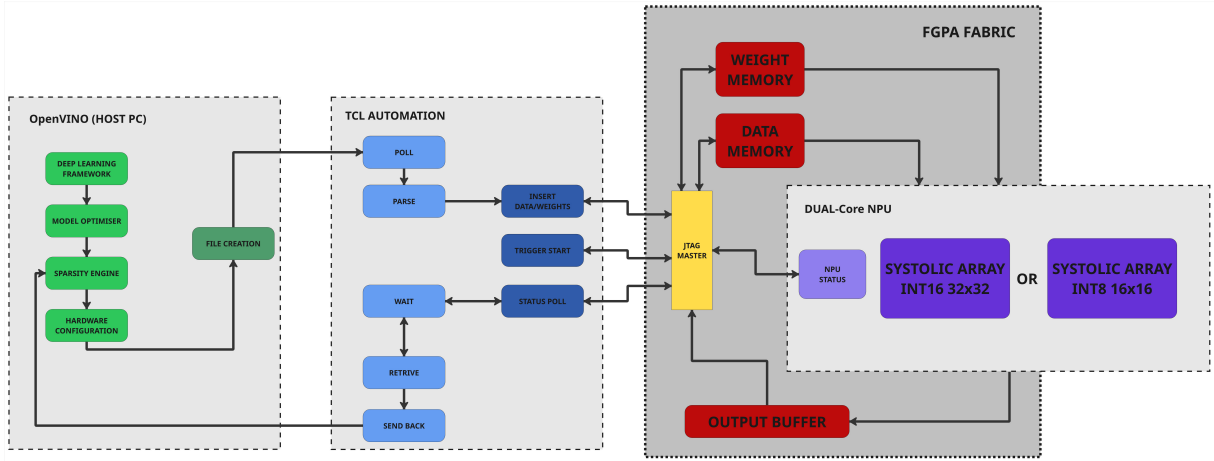


Figure 1 Complete System Diagram

4.1.1 System Dataflow

The process begins on the host computer, where the software stack extracts the metadata of the target CNN model. Unlike a standard inference pass, this pipeline divides the data; while weights are extracted statically from the OpenVINO intermediate Representation, activations are captured dynamically to account for runtime sparsity. These tensors are then quantised and subjected to the coordinated row-and-column removal algorithm, which compresses the sparse tiles into dense primitives.

4.1.2 Host-to-FPGA Pipeline

Communication is managed through a file-based handshake between the Python stack and a TCL automation layer.

- **Tile Metadata:** The effective dimensions and configuration flags are prepended to the matrix data as a binary header.
- **JTAG Transmission:** Data is streamed via the JTAG-to-Avalon Master Bridge directly into the Arria 10's on-chip M20K memory blocks.
- **Hardware Execution:** The NPU System Wrapper parses the header to select the optimal precision core and reconfigures the active processing region of the systolic array to match the compacted dimensions.

4.1.3 Hardware Architecture

The Arria 10 implementation utilises a dual-core NPU topology to balance numerical precision with logic efficiency. As shown in Figure 1, the fabric contains both an INT16 32x32 DSP-based core and an INT8 16x16 LUT-based core. By routing tiles based on their sparsity-stripped dimensions, the system ensures that the massive logic depth required for a 32x32 array is only utilised when necessary, while smaller or sparser tiles are processed by the high-throughput 8-bit core. The integration into the broader SoC fabric is achieved via the Avalon Memory-Mapped interconnect, ensuring bit-accurate data retrieval and write-back synchronised with the host's polling loop.

4.2 OpenVINO Software Stack on Host Computer

The software stack, implemented in Python, serves as the orchestration layer of the software-hardware co-design topology. The primary role behind it is to perform the high-level model analysis and matrix transformations that the FPGA cannot execute internally. The pipeline begins by importing pre-trained models from PyTorch, such as AlexNet, VGG16, ResNet50, and MobileNetV2, and converting them into the OpenVINO intermediate Representation (IR) format. This conversion is critical because it standardises diverse layer attributes, including kernel dimensions, strides, and padding into a hardware-agnostic XML and Bin format that the script can programmatically parse.

This stack uses graph augmentation to capture dynamic runtime data. The software clones the original OpenVINO model and inserts additional output nodes at the boundary of every discovered convolutional layer. By executing a single inference pass on the host CPU using a real input image, the software captures the exact post-ReLU activation tensors. This step is critical for exposing dynamic activation sparsity, which is image-dependent and cannot be predicted at synthesis time.

To prepare these high-level tensors for the systolic array, the stack performs a multi-step data preparation sequence. First, weights are extracted from the OpenVINO constant nodes and reshaped into 2D matrices. The stack then utilises an image-to-column (im2col) algorithm to unfold the 4D activation tensors into 2D matrices, ensuring the convolutional operation is mapped perfectly to a General Matrix-Matrix Multiplication (GEMM) format. To satisfy edge resource constraints and simplify FPGA arithmetic, the script applies linear quantisation by calculating a scaling factor based on the matrix's maximum absolute value of the matrix. This maps 32-bit floating-point values to signed 8-bit or 16-bit integers, depending on the target layer's required precision.

The central contribution of the software stack is the coordinated row- and column-removal algorithm for sparsity handling. Each processing tile is analysed to identify indices where entire rows in the data matrix or entire columns in the weight matrix consist of zeros. To maintain the mathematical integrity of the matrix product, the script performs synchronised stripping: if a column is removed from the activation matrix due to sparsity, the corresponding row must be removed from the weight matrix. This generates compacted, dense equivalents of the original sparse tiles, significantly reducing the number of Multiply-Accumulate (MAC) operations required.

Post matrix compacting, the software calculates the new effective dimensions (M, N, K) and utilises a configuration to route the task to the optimal hardware execution path. If the stripped dimensions M and N are both ≤ 16 , the stack selects the high-efficiency int8 16x16 Core (Config 1). For tiles exceeding these dimensions or requiring higher numerical precision to prevent accumulator overflow, it defaults to the Int16 32x32 Core (config 0). Furthermore, the stack packages these parameters and the dense matrix data into a binary payload. It writes a 4-byte header containing M, N, K, and the config flag, followed by the matrix data, into a tile.bin file. This file acts as the physical trigger for the TCL automation script to initiate the JTAG transmission to the FPGA hardware.

4.3 TCL Automation

To facilitate automated communication between the host computer and the FPGA, a custom TCL script was developed to manage the JTAG-to-Avalon Master interface. This script eliminates the need for manual memory initialisation and hardware resets, serving as a real-time bridge that

synchronises the Python software stack with the NPU hardware logic.

The script operates on a polling-and-triggering mechanism that relies on a physical file-based handshake. It continuously monitors the local directory for the tile.bin file generated by the Python pre-processing stage. Upon detection, the script performs a binary scan to parse the 4-byte header containing the M, N, K, and configuration parameters. This dynamic parsing is essential as it allows the script to determine whether to treat the payload as 8-bit or 16-bit data, ensuring the Avalon bus transactions match the intended hardware core's precision.

Once the payload is parsed, the script automates a precise hardware sequence via the JTAG service path. The orchestration follows a four-step execution flow:

- **System Reset:** The Script issues a write command to the NPU control register to clear any states from previous runs.
- **Memory Mapping:** The dense matrix data and configuration flags are written to their respective Avalon-MM base addresses.
- **Dimension Loading:** The calculated effective dimensions (M, N, K) are loaded into control registers, effectively resizing the active region of the systolic array for that specific tile.
- **Trigger and Poll:** The NPU is triggered by asserting a start flag. The script then enters a high-speed polling loop, reading the Done register until the hardware signals completion or a timeout threshold (500ms) is reached.

Following a successful computation, the script retrieves the results and performance metrics from the FPGA fabric. It reads the specific number of output words ($M \times N$) from the Output Buffer base address, along with the hardware cycle count, which is appended as the final word for performance analysis. Finally, the script deletes the original binary file to signal back to the Python environment that the hardware cycle is complete and ready for the next processing tile. This automated cycle enables the processing of entire neural network layers consisting of hundreds of tiles without user intervention.

4.4 Reconfigurable Dual Core NPU Architecture

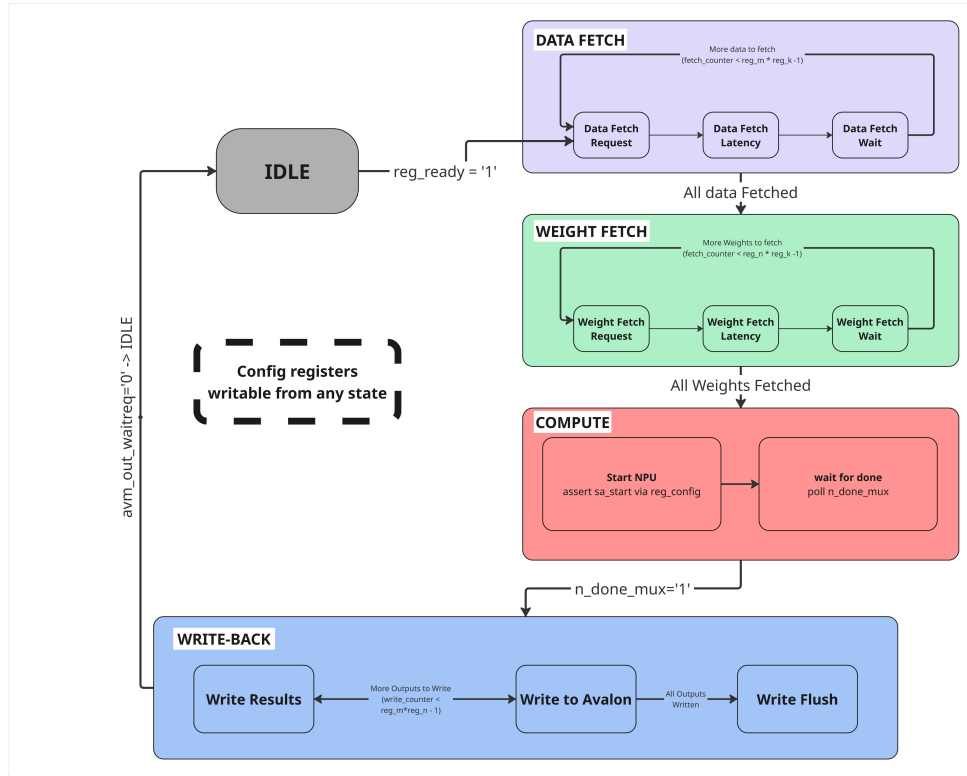


Figure 2 High-Level Diagram of the NPU System Wrapper With Encapsulated States

The main part of this project is the NPU System Wrapper, a sophisticated entity that manages the transition from high-level Avalon Memory-Mapped (Avalon-MM) transactions to the cycle-accurate staggering required by systolic arrays. This architecture is designed specifically to support reconfigurability, allowing the hardware to adapt its active computational region on a per-tile basis to exploit the sparsity identified by the software stack.

4.4.1 Dual-Core Precision Orchestration

The architecture is branched from the single-grid design by instantiating two parallel, specialised compute cores within the same wrapper logic. This dual-core approach allows the system to balance numerical dynamic range with hardware power efficiency. The Int16 32x32 Core serves as the high-precision engine, designed to handle large-scale tiles and layers with high weight density. By utilising 16-bit multipliers and 32-bit accumulators, this path provides the necessary numerical bands to prevent overflow during deep accumulation cycles ($k > 16$). The INT8 16x16 core is optimised for maximum throughput for minimal power. This core targets smaller, highly sparse tiles.

The selection between these cores is governed by a register configuration signal. Unlike a simple multiplexer, this configuration dynamically routes the global done signals and performance counters, ensuring that the Avalon Master interface perceives a single, unified NPU, regardless of the physical core executing the multiplication.

Table 1 NPU System Wrapper — Avalon Slave Register Map

Address	Bits	Register	R/W	Description
0x00	[0]	Ready Flag	R/W	Host sets to 1 to initiate a tile. Cleared by wrapper on completion.
0x04	[15:0]	M Dimension	R/W	Effective row dimension of stripped activation matrix.
0x08	[15:0]	N Dimension	R/W	Effective column dimension of stripped weight matrix.
0x0C	[15:0]	K Dimension	R/W	Shared inner dimension of the matrix product.
0x10	[0]	Core Config	R/W	0 = INT16 32×32 DSP core. 1 = INT8 16×16 LUT core.
0x14	[0]	Done Flag	R	Asserted by wrapper when selected core signals completion.
0x18	[31:0]	Debug State	R	Magic constant encoding current FSM state for JTAG inspection.
0x1C	[31:0]	Cycle Count	R	Latched cycle count from completed tile. Held until next tile starts.

Table 2 NPU System Wrapper — Avalon Master Memory Map

Base Address	Region	Description
0x21000	Activation Matrix	Source for activation (data) matrix elements. Addressed sequentially in row-major order. Elements packed as 32-bit words (INT8 lower byte, INT16 lower 16 bits).
0x22000	Weight Matrix	Source for weight matrix elements. Addressed sequentially in row-major order. Same packing convention as activation matrix.
0x23000	Output Buffer	Destination for M×N result elements. Written in row-major order as sign-extended 32-bit words.

4.4.2 Finite State Machine

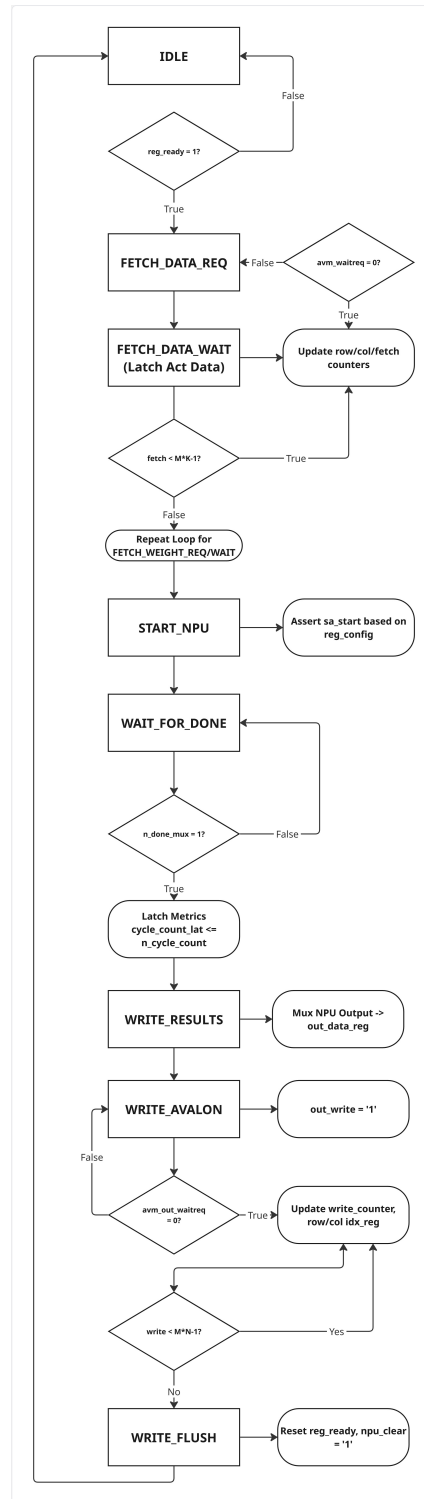


Figure 3 Algorithmic State Machine Diagram Of The FSM

The operational lifecycle of the NPU is governed by a 12-state FSM, which will be visualised in the accompanying ASM diagram 3.

During data extraction, the FSM acts as an Avalon-MM Master. It uses the fetch counter signal to generate sequential addresses and to retrieve matrix data from host memory. A critical hardware optimisation here is the FETCH_LATENCY state, as the on-chip BRAM and JTAG-to-Avalon bridge introduce synchronous delays; this state provides the necessary clock cycles

for data to latch into the internal input registers before the FSM increments the row and column indices.

The execution phase begins in the START_NPU state. Upon receiving the effective dimensions (M, N, K), the control unit generates the staggered wavefront. The FSM remains in the WAIT_FOR_DONE until the selected core asserts its completion flag. To ensure performance metrics are captured accurately, the wrapper latches the internal cycle_count into a persistent register the moment the done signal is detected, preventing the loss of timing data during the transition to the write-back phase.

As the physical dimensions of the systolic array scale to 32x32, the internal routing becomes a primary bottleneck. The output-stationary design requires a multiplexer tree that can select from 1024 different PEs. In initial synthesis runs, this resulted in a logic depth that limited Fmax to approximately 60 MHz.

To counteract this, the wrapper implements two critical physical-layer optimisations. Firstly was the Register Duplication. This is where a max_fanout attribute was applied to the row and column indices. This forced the Quartus synthesiser to duplicate the physical registers for these indices, placing copies locally within different DSP columns. This slashes the interconnect delay by reducing the physical wire length required to drive the massive PE selector tree. Secondly, there was the Pipelined Result Fetching. This is where a two-stage pipeline was added for result extraction. In the WRITE_RESULTS state, the wrapper places the row and column indices on the multiplexer bus and waits one full clock cycle. This 20ns window allows the long physical paths from the furthest PEs to settle. In the following cycle (WRITE_AVALON), the stable data is latched and committed to the Avalon bus.

The wrapper operates as both a slave to receive M,N,K parameters, and a Master to fetch data and write results. The write-back process utilises a lock-step handshake. The write counter and PE indices are only allowed to increment if the Avalon bus asserts a waitrequest '0' signal. If the bus is busy, the FSM stalls, holding the current PE data stable until the memory controller is ready. This synchronisation is important for bit-accuracy, ensuring that every result from the stripped MxN matrix is mapped to its correct physical address in the output buffer without skipping or duplicating values due to bus congestion.

To prevent the reset loop failures common in complex FSMs, a dedicated npu_clear signal was implemented. During the WRITE_FLUSH state, the system pulses this signal to clear all internal accumulators in the PE grid. Crucially, the IDLE state explicitly de-asserts this signal. This logic ensures that cores are released from hardware reset only when a fresh ready signal is received from the host, guaranteeing that each inference tile begins in a deterministic, clean state.

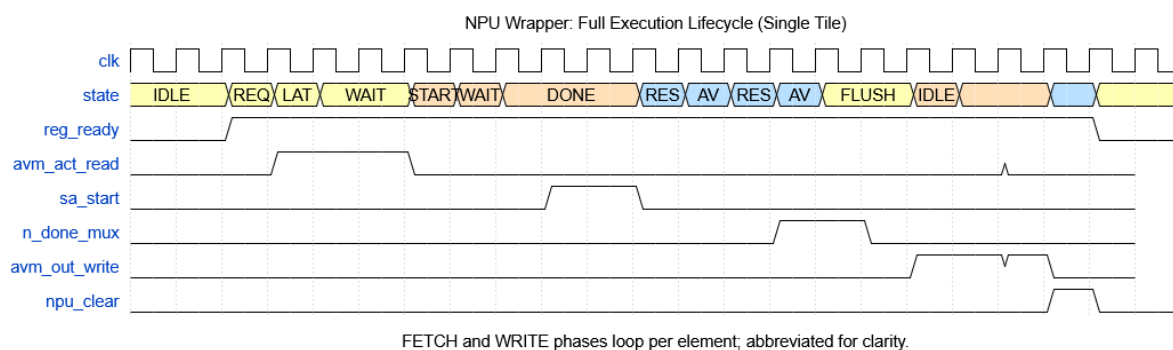


Figure 4 Full Execution Timing Diagram of the NPU

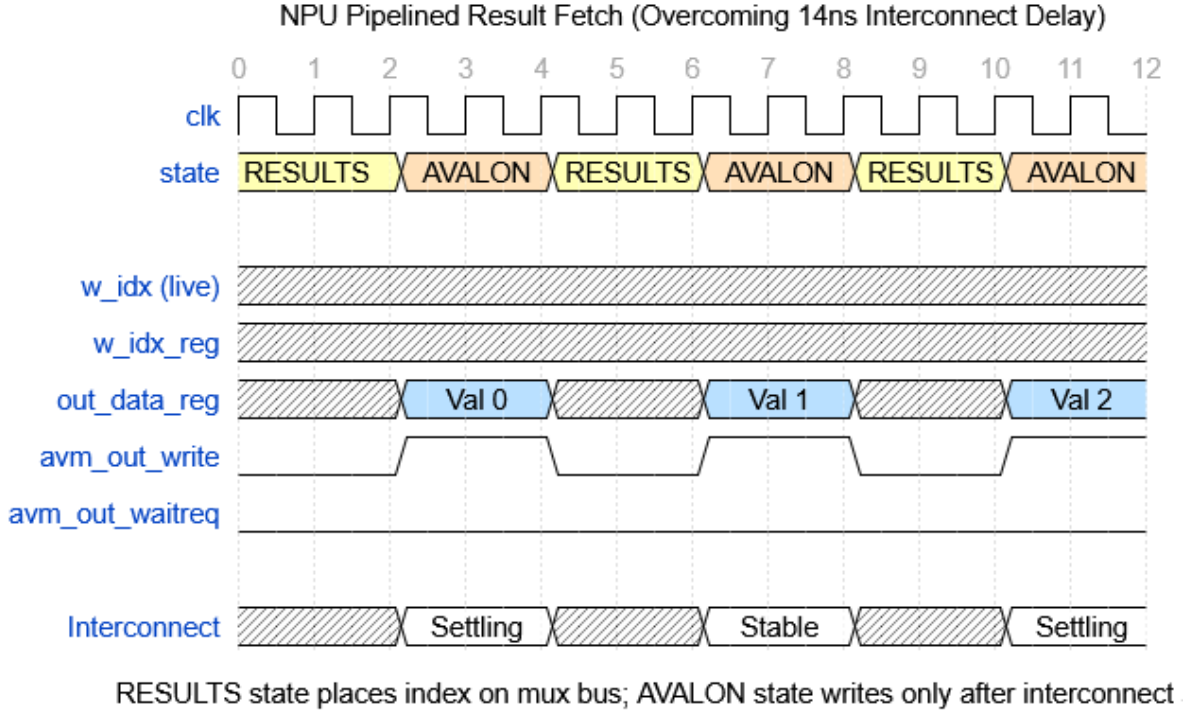


Figure 5 Timing Diagram of the Result Fetching

4.5 Verification and Validation Strategies

4.5.1 Pre-Silicone Performance Profiling and Software Validation

Before committing to a finalised hardware design, a comprehensive verification was executed using a modified Python script from the previous project. This stage served to validate the mathematical logic of the coordinated stripping algorithm and to provide a justification for a proper reconfigurable NPU.

The pipeline utilised the same PyTorch framework, the same quantisation and im2col transformations created in the previous project. However, some analysis was conducted on each tile and layer whilst the model ran for several images corresponding to different tile sizes. The profiling data revealed a critical performance bottleneck that dictated the final hardware design. As summarised in the layer analysis, a clear U-Shaped efficiency curve was observed across the network. The outer layers (Conv1 and Conv5) exhibited high spatial sparsity, leading to higher theoretical speedups. For example, Conv1 achieved a 2.3x speedup when using 8x8 tiles.

However, the analysis demonstrated an inverse relationship between physical tile size and sparsity exploitation. In the denser middle layers (Conv2 through Conv4), increasing the tile size to 32x32 led to a collapse in sparsity gain. This occurs because larger computational windows are statistically more likely to contain at least one non-zero value, preventing the row or column from being stripped. This finding provides that a static, systolic array would be fundamentally inefficient; a 32x32 array optimised for throughput to capture the significant cycle savings available in sparse layers.

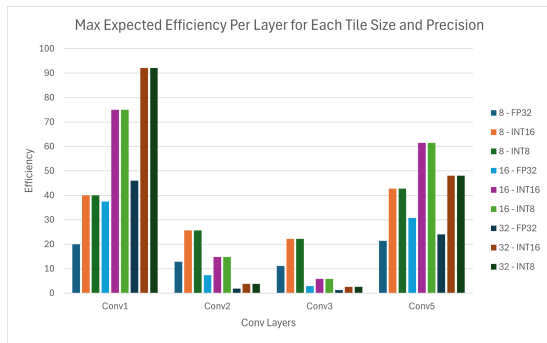


Figure 6 Theoretical Maximum Efficiency Achieved for each Tile and precision flexibility

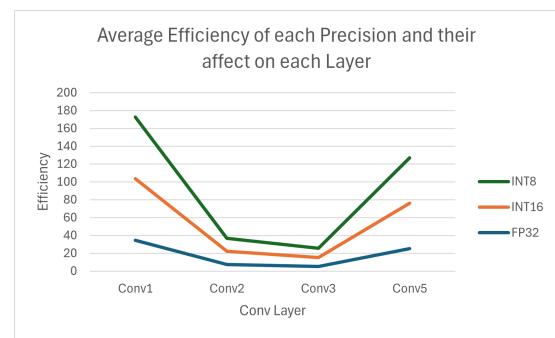


Figure 7 Theoretical Average efficiency achieved for each Quantisation precision

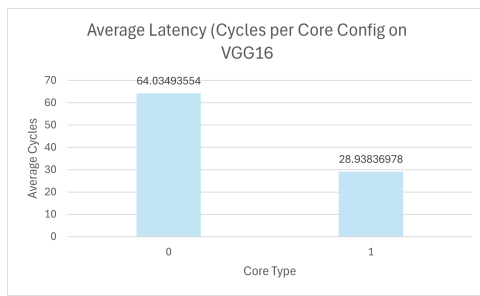
5. Results

5.1 NPU Performance Table

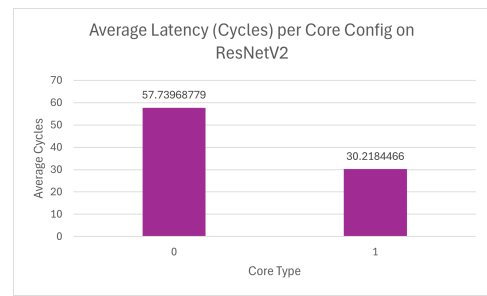
Table 3 Quartus Prime Synthesis and Implementation Results — Arria 10 (10AS066N3F40E2SG)

Category	Parameter	Value
Timing	Achieved F_{max}	70.04 MHz
	Constrained Clock (50 MHz met)	Yes
	Setup Slack	5.723 ns
	Hold Slack	0.020 ns
Power	Static (Leakage) Power	2,110.86 mW
	Dynamic Power	735.87 mW
	Total Thermal Power	2,846.73 mW
	Core Voltage	0.9 V
Resource Utilisation	ALMs Used	69,367 / 251,680 (27.6%)
	DSP Blocks (Reported)	1,286 / 1,687 (76.2%)
	DSP Blocks (Est. after merging)	≈ 645
	Total RAM Blocks (M20K)	7 / 2,131 (<1%)
	Dedicated Logic Registers	160,019
Device	Family	Intel Arria 10
	Part Number	10AS066N3F40E2SG
	Speed Grade	E2

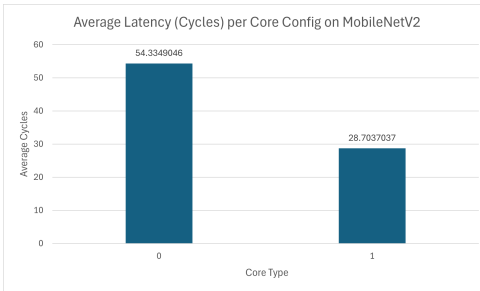
5.2 Results Collected from different CNN models



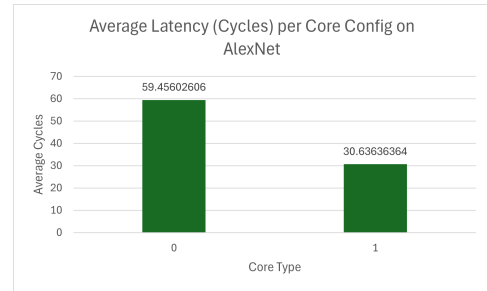
(a) VGG16



(b) ResNet50

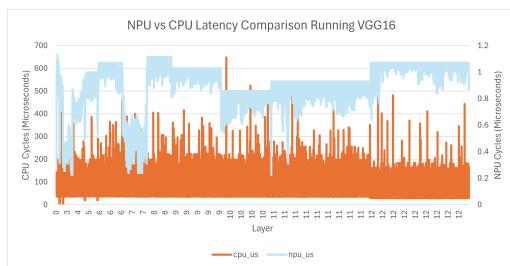


(c) MobileNetV2

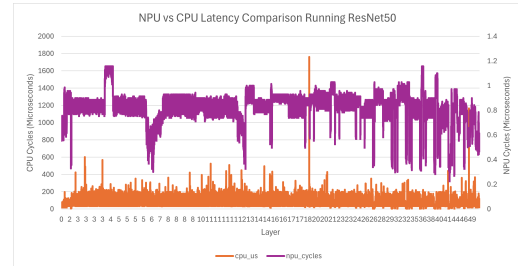


(d) AlexNet

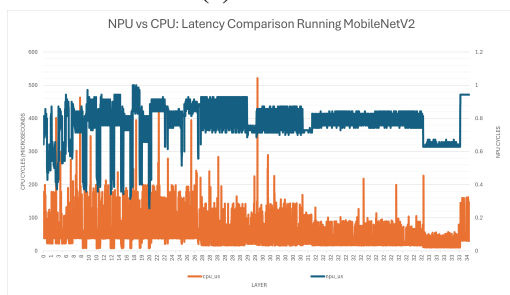
Figure 8 Average Latency per Core Configuration: Config 0 (32x32 DSP) vs. Config 1 (16x16 LUT).



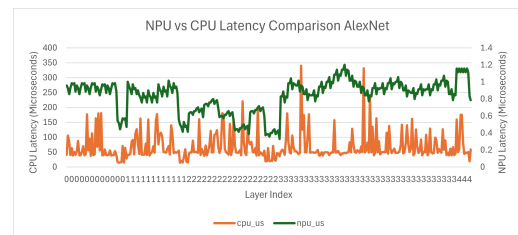
(a) VGG16



(b) ResNet50

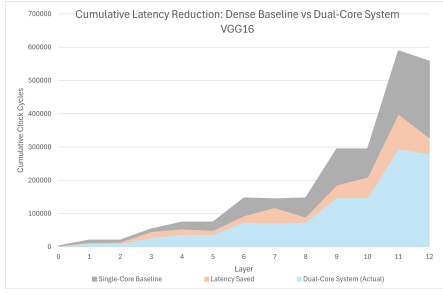


(c) MobileNetV2

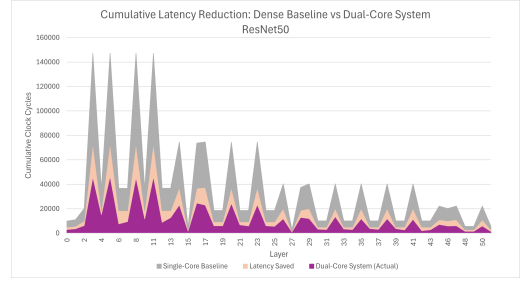


(d) AlexNet

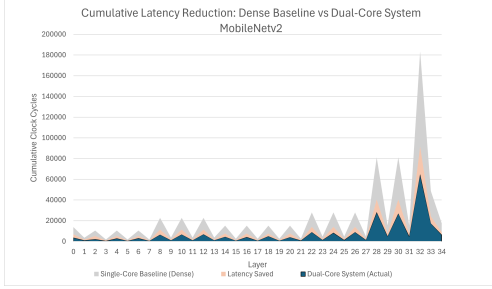
Figure 9 NPU vs. CPU Cycles per Layer comparison.



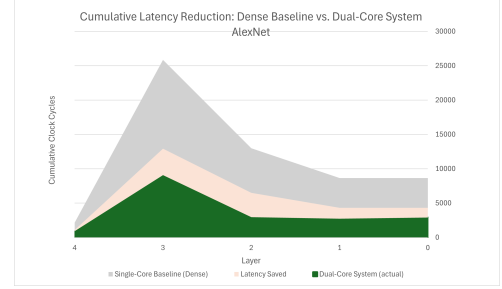
(a) VGG16



(b) ResNet50

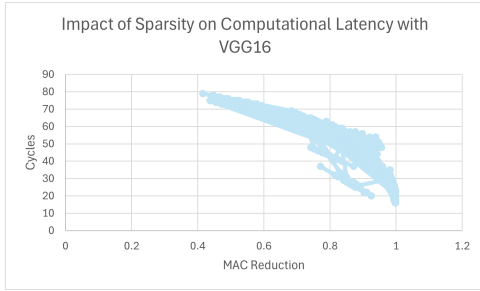


(c) MobileNetV2

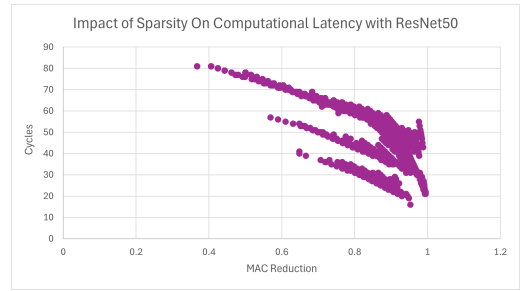


(d) AlexNet

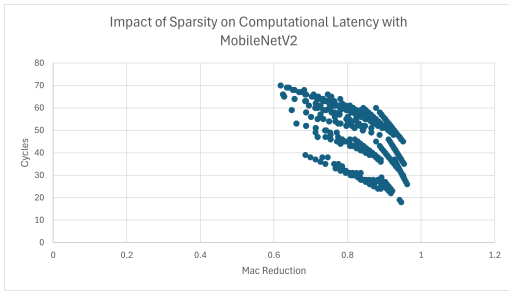
Figure 10 Latency Saved between Baseline and 2-Core NPU.



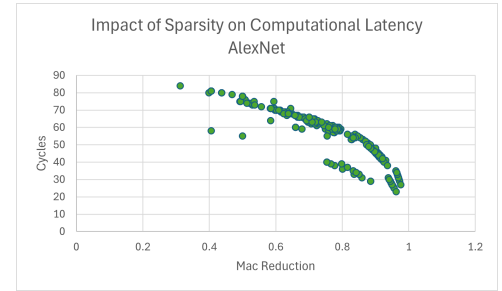
(a) VGG16



(b) ResNet50

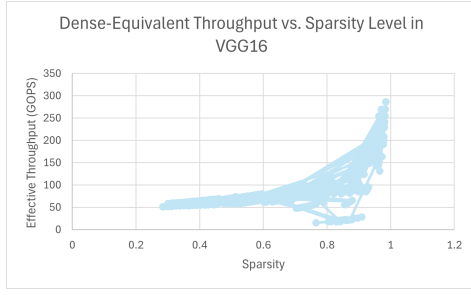


(c) MobileNetV2

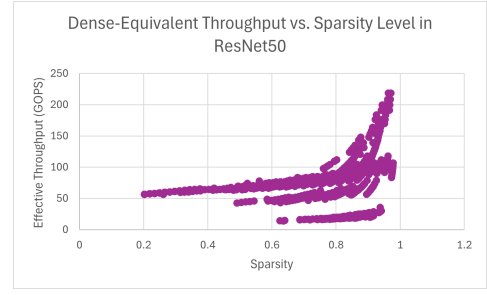


(d) AlexNet

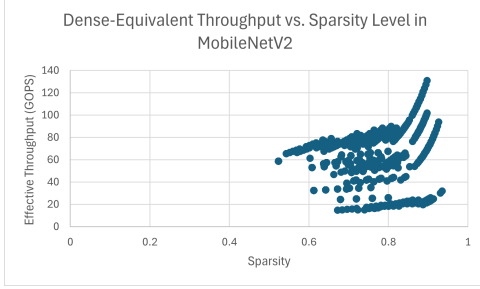
Figure 11 Impact of Sparsity on Computational Latency (Sparsity vs. Cycles).



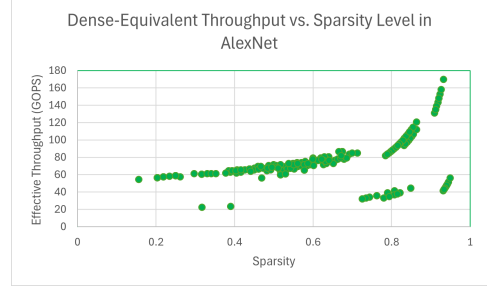
(a) VGG16



(b) ResNet50

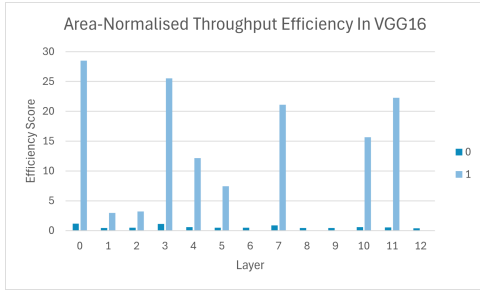


(c) MobileNetV2

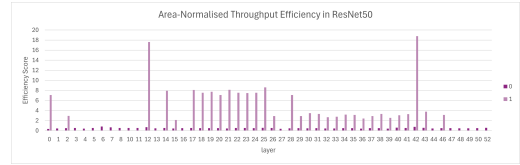


(d) AlexNet

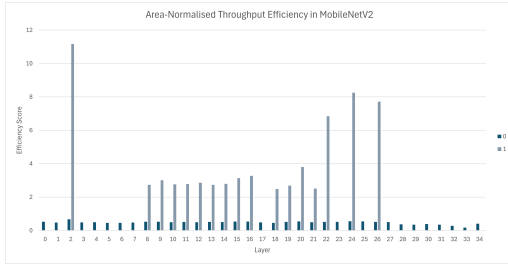
Figure 12 Dense-Equivalent Throughput (GOPS) vs. Sparsity Level.



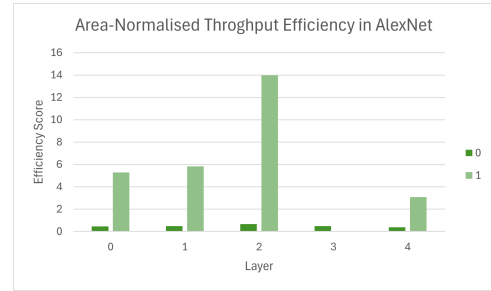
(a) VGG16



(b) ResNet50

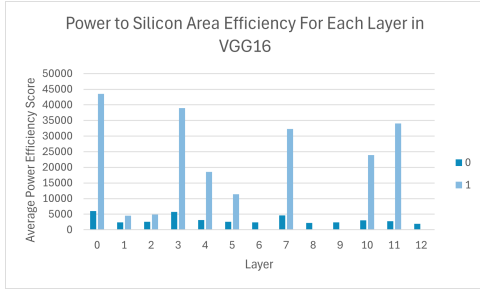


(c) MobileNetV2

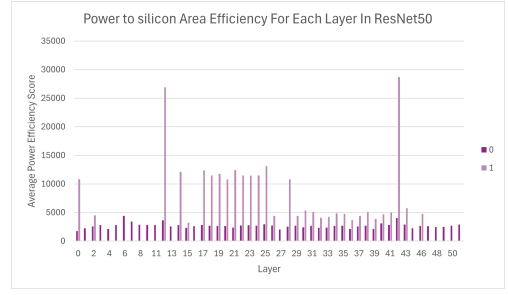


(d) AlexNet

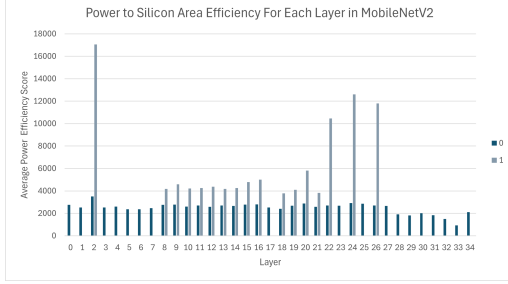
Figure 13 Normalized Area Efficiency (GOPS/Area) per Core Configuration.



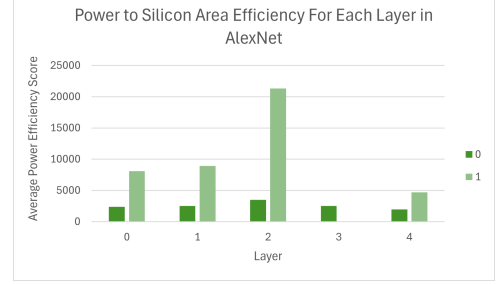
(a) VGG16



(b) ResNet50

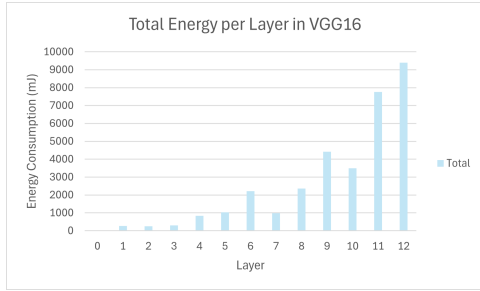


(c) MobileNetV2

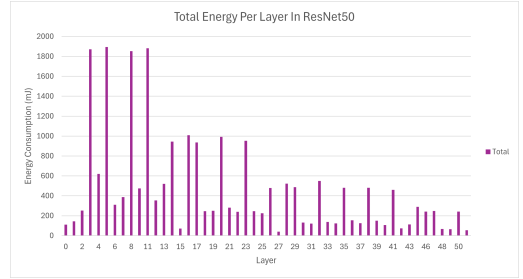


(d) AlexNet

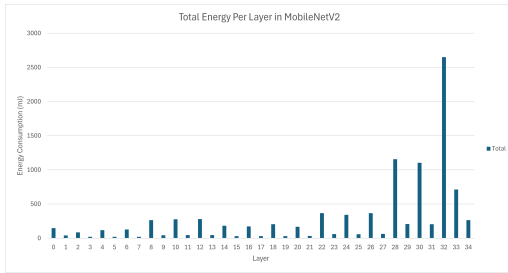
Figure 14 Power to Silicon Area Efficiency Score for Each Layer.



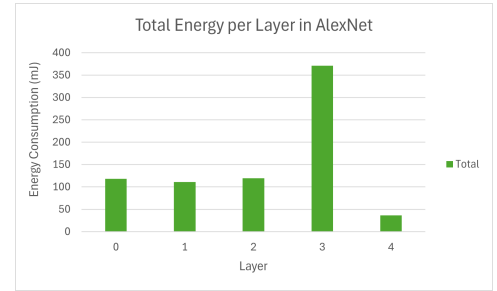
(a) VGG16



(b) ResNet50



(c) MobileNetV2



(d) AlexNet

Figure 15 Total Energy Consumption (mJ) per Layer.

Across all evaluated models, the hardware results demonstrate a consistent latency reduction when transitioning from the 32x32 DSP-based core (Config 0) to the 16x16 LUT-based core (Config 1). The data shows that Config 1 maintains a highly deterministic profile, with average per-tile latencies clustered around 30 cycles, compared to the 58-65 cycle range observed for Config 0. This performance gap is most pronounced in VGG16 and AlexNet, where the 16x16 core consistently halves the execution time for stripped tiles. Even in the more complex ResNet50 and MobileNetV2 architectures, Config 1 exhibits significantly lower variance and fewer outliers than Config 0, confirming that the dual-core controller effectively identifies and

offloads lower-complexity workloads to the more agile soft-logic core, thereby minimising execution overhead.

Across all models, the CPU vs NPU Cycles per layer results shown in Figure 9 show a significant disparity in the number of clock cycles required to execute each layer. The CPU cycle count is significantly higher and fluctuates widely across layers, often peaking at values 50x to 100x higher than the NPU's requirements. In contrast, the NPU maintains a consistently low cycle count across the entire network architecture. This data confirms that the NPU maintains a consistently low cycle count across the entire network architecture. This data confirms that the NPU hardware acceleration drastically reduces the computational overhead per layer compared to the CPU, providing a high-performance, deterministic execution baseline that the general-purpose processor cannot match.

For the latency gap per layer in Figure 10, the results quantify the direct cycle savings achieved by the dual-core architecture and the row-stripped algorithm. In every layer, the dual-core system consistently operates at a lower cycle count than the single-core baseline, with the latency saved representing a significant portion of the total execution time. For dense-heavy models like VGG16 and AlexNet, the latency gap is most substantial in the initial layers, whereas in MobileNetV2 and ResNet50, the savings remain high throughout the network. This data confirms that the system successfully exploits sparsity to bypass redundant computations, resulting in an average cycle reduction of 30% to 55% per layer compared to a standard dense execution.

For the impact of sparsity on tile latency across all the models, the results show a strong negative correlation between tile sparsity and execution cycles. As sparsity increases from 0% to nearly 90%, tile latency consistently drops from a peak of 85 cycles to approximately 35 cycles. This lower bound reflects the minimum tile dimensions achievable after stripping, as cycle counts are derived analytically from the effective dimensions M, N, and K using the theoretical systolic array latency formula. The data across all models confirm that the dual-core system successfully exploits some form of fine-grained sparsity, mostly structured sparsity, with MobileNetV2 and ResNet50 showing the most frequent utilisation of the high-sparsity, low-latency zone. This proves the NPU's ability to translate model-level pruning into predictable, real-time hardware speedups.

The results also confirm that the NPU successfully translates high sparsity into increased effective throughput. In VGG16 and AlexNet, throughput follows a clear upward trajectory as sparsity increases, peaking at approximately 120-130 GOPS for tiles with 80%-90% sparsity. This demonstrates that by skipping redundant zero-valued operations, the NPU delivers a performance that far exceeds its physical clock frequency. For MobileNetV2 and ResNet50, the data remain tightly clustered along this efficiency frontier, demonstrating that the hardware consistently maximises work per cycle regardless of the underlying model architecture. These findings validate that the coordinated row-stripping logic effectively bypasses the throughput bottlenecks typical of dense accelerators when encountering pruned neural networks.

This set of graphs evaluates the Normalised Area Efficiency, which is essentially the effective GOPS per Unit Area for both hardware configurations across all the models. The results demonstrate a massive efficiency disparity between the two cores, with Config 1, the 16x16 LUT-based one. Across all models, Config 0 maintains a baseline efficiency near 0.5, while Config 1 frequently peaks between 8.0 and 14.0 in sparse layers, such as in AlexNet and ResNet50. This indicates that while the larger DSP core provides high absolute throughput for dense tiles, it is significantly less resource-efficient than the smaller LUT core when processing stripped, sparse data. These findings confirm that the dual-core architecture successfully maximises

performance by offloading reduced workloads to the soft-logic core, which occupies approximately 82% less silicon area than the primary compute array.

Across all four models, the results reveal a massive efficiency advantage for Config 1 (16x16 LUT-based core) over Config 0 (32x32 DSP-based core). In VGG16 and AlexNet, Config 1 achieves efficiency scores peaking between 20,000 and 43,000, while Config 0 remains stagnant below 6000. This trend is consistent in ResNet50 and MobileNetV2, where the LUT-based maintains a significantly higher efficiency floor across all layers. The data confirms that for sparse workloads, utilising smaller soft-logic arrays yields a much higher return on silicon and power investment. By offloading stripped tiles to Config 1, the system bypasses the high area-power cost of the 1024 DSP blocks, maximising the overall throughput density of the NPU for deep, sparse neural network layers.

The energy consumption results for all the models quantify the NPU’s operational cost in millijoules (mJ). Given the Arria 10’s dominant static power floor of 2110.86 mW, the data reveals that total energy is primarily driven by execution time. In VGG16 and AlexNet, energy consumption is highest in the dense initial layers, indicated by the peaks near the right side of the graphs: 9000 mJ and 370 mJ, respectively. Conversely, in ResNet50 and MobileNetV2, the energy bars remain significantly lower and more distributed, with the majority of layers consuming under 1000mJ. This trend confirms that the row-stripping algorithm and dual-core switching effectively minimise the leakage window, ensuring that sparse workloads consume a fraction of the energy required by dense compute blocks.

6. Discussion

The results across VGG16, ResNet50, MobileNetV2, and AlexNet reveal patterns that go beyond validating architecture in isolation; they expose genuine architectural signatures of each model family through the NPU’s energy, latency, and efficiency profiles. Before interpreting these patterns, one important clarification about the data presentation is necessary. The OpenVINO inference graph traverses the network from the last layer to the first, meaning all per-layer figures in this report are plotted in reverse order, with the rightmost bars corresponding to the earliest convolutional layers. This inversion is critical for correctly reading the energy and efficiency distributions across all four models.

The claim of this work is not that the dual-core PU achieves peak performance on any single model, but rather that it adapts meaningfully to the structural characteristics of the model it receives, without any model-specific hardware configuration. The results across all four models collectively validate this. Each model exhibits a fundamentally different sparsity distribution, tile-size profile, and computational structure, yet in every case the routing logic correctly identifies which tiles belong to Config 0 and which to Config 1, and the resulting latency and efficiency profiles accurately reflect those architectural differences. This model-agnosticism, enabled by the OpenVINO IR extraction pipeline, is one of the more practically significant properties of the design; it demonstrates genuine reconfigurability in the sense that matters for edge deployment, the ability to handle diverse workloads without hardware modification along the way.

To properly interpret the area and power efficiency results in Figure 13 and 14, it is necessary to understand the fundamental difference between how the two cores occupy the Arria 10 fabric. Config 0 instantiates a 32x32 systolic array built from 1024 DSP blocks, along with approximately 42,000 ALMs. DSPs are hard IP, essentially physically etched multiplier blocks that are extremely fast for dense arithmetic but have a fixed silicon footprint regardless of

utilisation. When the stripping algorithm reduces a tile’s effective dimensions, the vast majority of those 1024 DSP blocks sit idle. At every layer, the dual-core system consistently operates at a lower cycle count than the single-core baseline, with the saved latency accounting for of Config 0 is paid in full even when a fraction of its compute capacity is needed. Config 1, by contrast, instantiates a 16x16 array built entirely from approximately 7600 ALMs in soft logic, with no DSP blocks involved. When normalised to a unified ALM-equivalent area unit, accounting for the fact that a DSP block represents significant functional silicon in ALM terms, Config 0’s total area footprint reaches approximately 140,000 ALM equivalents, compared to Config 1’s 7,600. This roughly 18x area difference is the primary reason Config 1’s normalised area efficiency in Figure 13 appears so disproportionately high: the denominator of the GOPS/Area calculation is an order of magnitude smaller, meaning every unit of effective throughput Config 1 produces is credited against a fraction of the silicon cost that Config 0 would require for the same tile. The power picture compounds this further. The Quartus power analysis estimates Config 0’s dynamic power at approximately 650 mW when its DSP array is actively switching, while Config 1 reduces this to approximately 100 mW by leaving the DSP blocks idle and switching only soft logic. Both cores share the Arria 10’s static leakage floor of 2,110.86 mW, which dominates total power consumption and means execution time is the primary driver of energy per layer. The power-to-area efficiency metric in Figure 14 penalises Config 0 on both dimensions simultaneously, its high dynamic power and its large silicon footprint, while Config 1 benefits from parking the DSPs entirely.

VGG16 achieves the highest peak dense-equivalent throughput of all evaluated models in Figure 12, approaching 300 GOPS at high sparsity levels. This figure, however, requires careful interpretation. The dense-equivalent GOPS metric counts zero-valued operations as if they were performed, meaning high GOPS is partly a consequence of high sparsity rather than high raw compute throughput. In VGG16’s case, the large spatial feature maps in its earliest layers, visible as the tallest energy bars on the right of Figure 15, peaking near 9000mJ, generate the broadest activation matrices, which in turn provide the most surface area for the stripping algorithm to find and remove zero-valued rows and columns. The resulting high GOPS reflects how effectively the NPU exploited that sparsity structure rather than indicating that VGG16 is inherently the fastest model to process. As the network deepens and feature maps shrink, energy drops sharply to the left in Figure 15, and Config 1 handles an increasing proportion of tiles, confirming that VGG16’s computational weight is front-loaded in its earliest spatial layers, exactly as its architecture predicts.

AlexNet achieves the highest normalised area efficiency scores for Config 1 in Figure 13, peaking around 14-15, despite being the smallest and shallowest network evaluated, with only five convolutional layers. The tiles that route to Config 1 after stripping tend to be those where dimensional reduction has been most aggressive, producing a high GOPS per ALM equivalent return from the 16x16 soft-logic core at minimal silicon cost. The small number of distinct data points visible for AlexNet in Figure 12 reflects this shallow structure; each layer’s tile distribution is individually distinct rather than blended across many layers, making the relationship between sparsity and computed latency cleaner than in deeper networks. AlexNet demonstrates that the dual-core strategy scales down effectively: even in a compact network, the routing logic identifies the layers where Config 1 delivers disproportionate efficiency gains and offloads them accordingly.

The most architecturally revealing finding across all four models is the energy distribution in ResNet50, shown in Figure 15. Unlike VGG16, AlexNet, and MobileNetV2, which all show their highest energy consumption on the right side of the graph corresponding to their earliest and spatially largest layers, ResNet50’s energy bars peak on the left, in its deepest layers. This

is a direct consequence of ResNet50’s bottleneck block design. The network’s initial stem and early residual blocks operate at shallow channel depths, keeping their computational cost modest. As the network progresses through its four stages, the channel dimensions expand from 256 to 512, and from 1024 to 2048 in the deepest blocks. These channel-rich bottleneck layers are ResNet50’s most computationally demanding, and because OpenVINO traverses the graph in reverse, they appear on the left of the energy plot. The NPU’s output is not unexpected; it is accurately demonstrating ResNet50’s architectural decision to distribute computational cost toward depth rather than spatial scale. Figure 14 reinforces this: Config 1’s power-to-area efficiency spikes most sharply in ResNet50’s deeper layers, because those bottleneck blocks, despite their channel density, still generate tiles with sufficient structured sparsity after stripping for the soft-logic core to handle them with outsized efficiency relative to its 7600 ALM footprint, a footprint that would have been entirely idle had Config 0 processes those tiles instead.

MobileNetV2 presents the most instructive boundary case of the four models. Its total energy consumption is by far the lowest and most uniformly distributed, consistent with its design as a mobile optimised architecture. However, Figure 13 shows that its normalised area efficiency for Config 1 is also the most suppressed, remaining flat and low across all the layers in a way that distinguishes it clearly from the other three models. The reason lies in MobileNetV2’s reliance on depthwise separable convolutions. The pointwise 1x1 convolutions that follow each depthwise stage yield small tile dimensions after the im2col transformation, which often result in Config 1 based on their M and N values. However, these 1x1 tiles carry lower structured sparsity than spatial convolutions because the spatial neighbourhood that ReLU zeros out in a standard convolution does not exist in a pointwise operation, and there is no spatial structure for the row-and-column removal algorithm to exploit. The result is tiles that are small but relatively dense, limiting how much the stripping algorithm can remove and consequently how high the effective GOPS can climb. The tightly clustered sparsity band visible for MobileNetV2 in Figure 12 is narrower than any other model; essentially, the quantitative signature of this constraint. MobileNetV2, therefore, defines the practical boundary of the structured sparsity approach: when convolutions are already decomposed into their most minimal spatial form, the stripping algorithm has less redundancy to find, and the efficiency gains from dual-core routing are correspondingly more modest. That the system still handles MobileNetV2 correctly and routes its tiles appropriately, rather than misclassifying them or stalling, confirms the robustness of the routing logic even when the model does not present favourable conditions for exploitation.

Considered together, the four model profiles form a coherent argument for the design’s reconfigurability. VGG16’s front-loaded spatial cost, ResNet50’s depth-shifted bottleneck budget, AlexNet’s concentrated sparsity in a shallow structure and MobileNetV2’s structurally constrained sparsity each yield distinct routing contributions and efficiency profiles, yet the dual-core controller handles both without further modification. The GOPS metric, read carefully as a measure of how effectively each model’s sparsity was exploited rather than as a raw throughput figure, confirms that the system extracts meaningful gains wherever the model’s architecture permits and degrades gracefully where it does not. The area and power efficiency results make the underlying reason concrete: Config 1’s 7600 ALM footprint versus Config 0’s approximately 140,000 ALM-equivalent footprint means that every correct routing decision forward the soft-logic core is not merely a latency saving but a proportional reduction in dynamic power draw. This is the practical definition of reconfigurability for edge inference, not peak performance on a single optimised workload, but consistent and correct adaptation across the diverse range of models that real deployments demand, extracting the maximum return from available silicon at every opportunity the sparsity structure of the model allows.

7. Future Work

The current implementation uses the JTAG-to-Avalon Master Bridge as a functional verification interface, which, while sufficient for establishing bit-accurate hardware validation across four vision models imposes a significant communication bottleneck that prevents the system from operating at its full computational potential. Two primary upgrade pathways have been identified that would eliminate this constraint and transition the system from a hardware-verified prototype to a production-capable inference platform.

7.1 HPS-Based Deployment with On-Device OpenVINO

The most architecturally coherent upgrade pathway for this system is the migration of the OpenVINO host stack from an external laptop to the Arria 10's integrated Hard Processor System. The Arria 10 SoC incorporates a dual-core ARM Cortex-A9 HPS connected to the FPGA fabric via AMBA AXI-3 compliant HPS-to-FPGA bridges, supporting data widths of up to 128 bits with direct access to the DDR4 memory subsystem [24]. By booting a Linux kernel on the HPS cores and deploying the OpenVINO inference engine natively, the tile dispatch pipeline could communicate with the NPU accelerator entirely on-chip via the high-speed H2F bridge, eliminating the external cable latency and bit-serial overhead that currently limits end-to-end throughput. This was the original motivation for adopting OpenVINO as the software stack in this project, as the toolkit is specifically designed to support heterogeneous FPGA/CPU deployment on Intel SoC platforms [25]. Under this architecture, the ARM cores would handle model loading, IR graph traversal, im2col transformation, quantisation, and sparsity analysis, while dispatching compacted tiles directly to the dual-core NPU via memory-mapped registers over the H2F bridge. This would remove the file-based handshake mechanism entirely and replace it with a deterministic, low-latency dispatch loop running entirely within the SoC, with the primary remaining bottleneck being DDR4 bandwidth rather than communication protocol overhead.

7.2 PCIe-Based High-Bandwidth Host Interface

An alternative pathway for deployment scenarios where a host PC acts as the inference orchestrator is the integration of a PCIe Gen3 interface between the host system and the Arria 10 FPGA. The Arria 10 includes hard PCIe IP supporting Gen3 speeds, and research by Manavar et al. demonstrates that configuring a PCIe interface in streaming mode using the XDMA IP eliminates memory copy overhead entirely, achieving throughput bounded only by the synthesis frequency of the accelerator rather than the protocol overhead of the communication interface [23]. Applied to this system, a PCIe streaming interface would allow the OpenVINO host stack running on an external PC to blast raw tile data directly into the NPU's input buffers at Gbps-level rates, reducing the per-tile transfer time from the tens of milliseconds currently imposed by JTAG to the sub-microsecond range. This would allow the hardware cycle count measurements already captured by the NPU to become representative of true end-to-end inference latency rather than communication-dominated system latency, and would enable real-time inference benchmarking across the four vision models at their full hardware throughput. This pathway is particularly relevant for data centre or workstation deployment scenarios where the FPGA is installed as a PCIe accelerator card and the host CPU manages model orchestration, mirroring the architecture evaluated by Manavar et al. for ML inference workloads.

8. Conclusion

This work presented the design, implementation, and evaluation of a reconfigurable dual-core NPU on an Intel Arria 10 SoC FPGA, targeting energy-efficient CNN inference at the edge. By extending a prior simulation-based co-design approach into a hardware-verified multi-model system, the design demonstrated that a heterogeneous INT16 32x32 DSP-based and INT8 16x16 LUT-based compute architecture, driven by a runtime sparsity-aware tile routing strategy, can adapt meaningfully to the characteristics of diverse CNN models without hardware modification. Evaluated across VGG16, ResNet50, MobileNetV2, and AlexNet using an OpenVINO-based software pipeline, the system achieved an average cycle reduction of 30-55% per layer over a dense single-core baseline, with the LUT-based core delivering normalised area efficiency scores up to 28x higher than the DSP-based core for sparse workloads. The energy profiles across all four models confirmed that the dual-core routing strategy correctly identifies and exploits model-specific sparsity distributions, validating the system as a practical and generalisable solution for accelerating sparse CNNs on FPGA hardware. The primary remaining constraint is the JTAG-based communication interface, which limits end-to-end throughput independent of the compute fabric’s capabilities; migrating to HPS-based on-device deployment or PCIe streaming represents the natural next step toward a production-capable inference platform.

References

- [1] P. Chhabra **and** K. Sampang, “Project #43: Reconfigurable Neural Processing Unit (NPU) for Energy-Efficient AI at the Edge,” en, **october** 2025.
- [2] J. Cong **and** B. Xiao, “Minimizing Computation in Convolutional Neural Networks,” en, **in** *Artificial Neural Networks and Machine Learning – ICANN 2014* S. Wermter **and** others, editors, Series Title: Lecture Notes in Computer Science, **volume** 8681, Cham: Springer International Publishing, 2014, **pages** 281–290, ISBN: 978-3-319-11178-0 978-3-319-11179-7. DOI: [10.1007/978-3-319-11179-7_36](https://doi.org/10.1007/978-3-319-11179-7_36). **url**: http://link.springer.com/10.1007/978-3-319-11179-7_36.
- [3] J. Zhang **and** J. Li, “Improving the Performance of OpenCL-based FPGA Accelerator for Convolutional Neural Network,” **in** *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* **jourser** FPGA ’17, New York, NY, USA: Association for Computing Machinery, **february** 2017, **pages** 25–34, ISBN: 978-1-4503-4354-1. DOI: [10.1145/3020078.3021698](https://doi.org/10.1145/3020078.3021698). **url** seen 18 **october** 2025. **url**: <https://doi.org/10.1145/3020078.3021698>.
- [4] J. Wu, “Introduction to convolutional neural networks,” *National Key Lab for Novel Software Technology. Nanjing University. China*, **jourvol** 5, **number** 23, **page** 495, 2017.
- [5] NVIDIA, *Matrix Multiplication Background User’s Guide*, en. **url** seen 18 **october** 2025. **url**: <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>.
- [6] E. Qin **and** others, “SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training,” en, **in** *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)* San Diego, CA, USA: IEEE, **february** 2020, **pages** 58–70, ISBN: 978-1-7281-6149-5. DOI: [10.1109/HPCA47549.2020.00015](https://doi.org/10.1109/HPCA47549.2020.00015). **url**: <https://ieeexplore.ieee.org/document/9065523/>.
- [7] Purwono, A. Ma’arif, W. Rahmانيar, H. I. K. Fathurrahman, B. Setiawan **and** H. Wibowo, “Understanding of Convolutional Neural Network (CNN): A Review,” *International Journal of Robotics and Control Systems*, **jourvol** 2, **number** 4, **pages** 739–748, 2022. DOI: [10.31763/ijrcs.v2i4.888](https://doi.org/10.31763/ijrcs.v2i4.888).
- [8] H. H. Tan **and** K. H. Lim, “Vanishing Gradient Mitigation with Deep Learning Neural Network Optimization,” **in** *2019 7th International Conference on Smart Computing & Communications (ICSCC)* **june** 2019, **pages** 1–4. DOI: [10.1109/ICSCC.2019.8843652](https://doi.org/10.1109/ICSCC.2019.8843652). **url**: <https://ieeexplore.ieee.org/document/8843652>.
- [9] H. T. Kung **and** C. E. Leiserson, “Systolic arrays (for VLSI),” **in** *Sparse Matrix Proceedings 1978* Society for industrial **and** applied mathematics Philadelphia, PA, USA, **volume** 1, 1979, **pages** 256–282.
- [10] H.-T. Kung, *Why systolic architecture?* Design Research Center, Carnegie-Mellon University Pittsburgh, PA, USA, 1982.
- [11] G. I/O, *An in-depth look at Google’s first Tensor Processing Unit (TPU)*, en, **may** 2017. **url** seen 14 **october** 2025. **url**: <https://cloud.google.com/blog/products/ai-machine-learning/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu>.

- [12] W. Sun, D. Liu, Z. Zou, W. Sun, S. Chen and Y. Kang, “Sense: Model-Hardware Codesign for Accelerating Sparse CNNs on Systolic Arrays,” en, *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, **jourvol** 31, **number** 4, **pages** 470–483, **april** 2023, Publisher: Institute of Electrical and Electronics Engineers (IEEE), ISSN: 1063-8210, 1557-9999. DOI: [10.1109/tvlsi.2023.3241933](https://doi.org/10.1109/tvlsi.2023.3241933). **url:** <https://ieeexplore.ieee.org/document/10043636/>.
- [13] X. He and others, “Sparse-TPU: adapting systolic arrays for sparse matrices,” en, *in Proceedings of the 34th ACM International Conference on Supercomputing* Barcelona Spain: ACM, **june** 2020, **pages** 1–12, ISBN: 978-1-4503-7983-0. DOI: [10.1145/3392717.3392751](https://doi.org/10.1145/3392717.3392751). **url:** <https://dl.acm.org/doi/10.1145/3392717.3392751>.
- [14] A. Parashar and others, “SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks,” en, **may** 2017, arXiv:1708.04485 [cs]. DOI: [10.48550/arXiv.1708.04485](https://doi.org/10.48550/arXiv.1708.04485). **url:** <http://arxiv.org/abs/1708.04485>.
- [15] P. Palacios, R. Medina, J.-L. Rouas, G. Ansaloni and D. Atienza, “Systolic arrays and structured pruning co-design for efficient transformers in edge systems,” *in Proceedings of the Great Lakes Symposium on VLSI 2025* 2025, **pages** 320–327.
- [16] J.-S. Park and others, “9.5 A 6K-MAC Feature-Map-Sparsity-Aware Neural Processing Unit in 5nm Flagship Mobile SoC,” en, *in 2021 IEEE International Solid-State Circuits Conference (ISSCC)* San Francisco, CA, USA: IEEE, **february** 2021, **pages** 152–154, ISBN: 978-1-7281-9549-0. DOI: [10.1109/ISSCC42613.2021.9365928](https://doi.org/10.1109/ISSCC42613.2021.9365928). **url:** <https://ieeexplore.ieee.org/document/9365928/>.
- [17] S. Kim, S. Cho, E. Park and S. Yoo, “FPGA Prototyping of Systolic Array-based Accelerator for Low-Precision Inference of Deep Neural Networks,” en, *in 2021 IEEE International Workshop on Rapid System Prototyping (RSP)* Paris, France: IEEE, **october** 2021, **pages** 1–7, ISBN: 978-1-6654-6956-2. DOI: [10.1109/RSP53691.2021.9806200](https://doi.org/10.1109/RSP53691.2021.9806200). **url:** <https://ieeexplore.ieee.org/document/9806200/>.
- [18] J. S. Fata, W. M. Elmannai and K. Elleithy, “Balancing Performance and Cost—FPGA-Based CNN Accelerators for Edge Computing: Status Quo, Key Challenges, and Prospective Innovations,” en, *IEEE Access*, **jourvol** 13, **pages** 142 852–142 877, 2025, ISSN: 2169-3536. DOI: [10.1109/ACCESS.2025.3594348](https://doi.org/10.1109/ACCESS.2025.3594348). **urlseen** 9 december 2025. **url:** <https://ieeexplore.ieee.org/document/11105416/>.
- [19] M. Bharathi and Y. J. M. Shirur, “Efficiency Evaluation of Scalable Multiply and Accumulate Architectures in DSP: A Comparative Study of LUT Based and LUT-Less Based Approaches,” en, *in 2024 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)* Bangalore, India: IEEE, **january** 2024, **pages** 1–5, ISBN: 979-8-3503-0641-5. DOI: [10.1109/IITCEE59897.2024.10467265](https://doi.org/10.1109/IITCEE59897.2024.10467265). **urlseen** 3 april 2026. **url:** <https://ieeexplore.ieee.org/document/10467265/>.
- [20] Y. Cao, H. Jiang, H. Li, Y. Tang, D. Shi and G. Zhao, “HMSA: High-Performance Heterogeneous Mixed-Precision CNN Systolic Array Accelerator on FPGA,” en, *ACM Transactions on Embedded Computing Systems*, **jourvol** 24, **number** 5s, **pages** 1–27, **november** 2025, ISSN: 1539-9087, 1558-3465. DOI: [10.1145/3759458](https://doi.org/10.1145/3759458). **urlseen** 3 april 2026. **url:** <https://dl.acm.org/doi/10.1145/3759458>.
- [21] Y. Shao and others, “A Configurable Accelerator for CNN-Based Remote Sensing Object Detection on FPGAs,” en, *IET Computers & Digital Techniques*, **jourvol** 2024, **number** 1, S. K. Srinivasan, **editor**, **page** 4 415 342, **january** 2024, ISSN: 1751-8601, 1751-861X. DOI: [10.1049/2024/4415342](https://doi.org/10.1049/2024/4415342). **urlseen** 3 april 2026. **url:** <https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/2024/4415342>.

- [22] A. Parameshwara **and** S. H. Mokashi, *FPGA-Accelerated RISC-V ISA Extensions for Efficient Neural Network Inference on Edge Devices*, en, arXiv:2511.06955 [cs], **november** 2025. DOI: [10.48550/arXiv.2511.06955](https://doi.org/10.48550/arXiv.2511.06955). **urlseen** 26 **november** 2025. **url**: <http://arxiv.org/abs/2511.06955>.
- [23] P. Manavar, M. Nambiar, N. Sumeet, R. Singhal, S. Choudhary **and** A. Pandit, “Experience with PCIe streaming on FPGA for high throughput ML inferencing,” en,
- [24] I. Corp., “Arria® 10 Device Overview,” en,
- [25] F. Z. Guerrouj, M. Abouzahir, M. Ramzi **and** E. M. Abdali, “Analysis of the acceleration of deep learning inference models on a heterogeneous architecture based on OpenVINO,” en, **in** *2021 4th International Symposium on Advanced Electrical and Communication Technologies (ISAECT)* Alkhobar, Saudi Arabia: IEEE, **december** 2021, **pages** 01–05, ISBN: 978-1-6654-3773-8. DOI: [10.1109/ISAECT53699.2021.9668607](https://doi.org/10.1109/ISAECT53699.2021.9668607). **urlseen** 3 **april** 2026. **url**: <https://ieeexplore.ieee.org/document/9668607/>.